

Test Evidence/Schema SQL

Campus Equipment Booking API (Hono + Cloudflare D1)

Student name	<i>Wacharaphong Sutthiboriban</i>
Student ID	<i>6731503032</i>
Platform	Cloudflare Workers + D1 (remote, deployed)
Framework	Hono (TypeScript)
Base API URL	<code>https://booking-api.student-projects.workers.dev/api</code>

1. Deployment Proof

1.1 Remote database setup — npm run db:remote

```
PS D:\project\booking-api> npm run db:remote
>> npm run deploy

> db:remote
> wrangler d1 execute booking-db --remote --file=schema.sql
```

 wrangler 4.147.0

Resource location: remote

✓ ⚠ This process may take some time, during which your D1 database will be unavailable to serve queries.
Ok to proceed? ... yes

- 🌀 Executing on remote database booking-db (a96ecbde-82bc-43c6-9df0-b826df14f69a):
- 🌀 To execute on your local development database, remove the --remote flag from your wrangler command.

Note: if the execution fails to complete, your DB will return to its original state and you can safely retry.

- 🌀 File already uploaded. Processing.
- 🌀 Starting import...
- 🌀 Processed 6 queries.
- 🌊 Executed 6 queries in 4.20ms (18 rows read, 13 rows written)

Database is currently at bookmark 00000002-00000008-000050fc-35ba38e7e9e2fe71674e2a9e4754902a.

Total queries executed	Rows read	Rows written	Database size (MB)
6	18	13	0.03

1.2 Worker deployment — npm run deploy

 wrangler 4.147.0

Total Upload: 61.97 KiB / gzip: 15.70 KiB

Your Worker has access to the following bindings:

Binding	Resource
env.DB (booking-db)	D1 Database

Uploaded booking-api (7.63 sec)

Deployed booking-api triggers (2.37 sec)

<https://booking-api.student-projects.workers.dev>

Current Version ID: c172c432-7d48-4a1e-a691-836210c95300

```
PS D:\project\booking-api> $BASE = "https://booking-api.student-projects.workers.dev/api"
```

1.3 PowerShell variable used

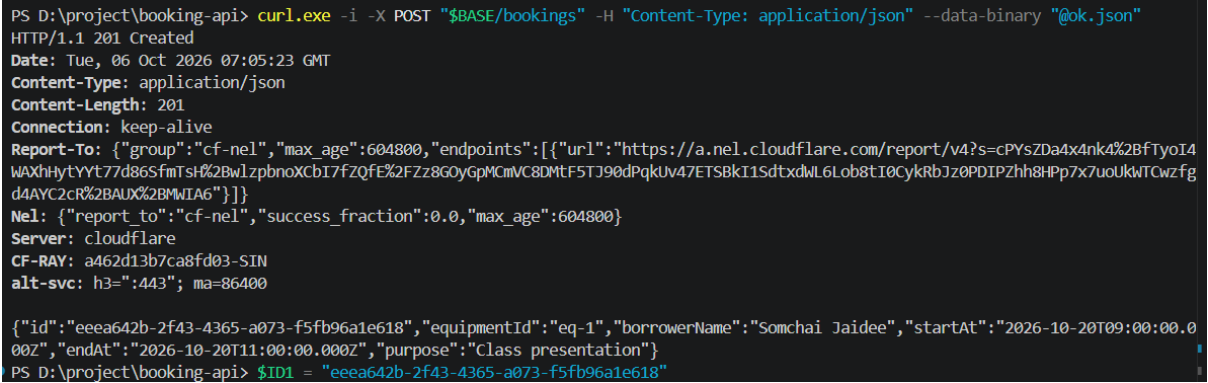
\$BASE = "https://booking-api.student-projects.workers.dev/api"

2. Test Cases

Test 1 — List equipment (core)

Command	curl.exe -i "\$BASE/equipment"
Expected	200 OK — Returns the 3 seeded equipment records.
Screenshot	 <pre>PS D:\project\booking-api> curl.exe -i "\$BASE/equipment" HTTP/1.1 200 OK Date: Tue, 06 Oct 2026 06:52:54 GMT Content-Type: application/json Content-Length: 178 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=qwMgMN0g6holwHau3ZPhioP5ke2ZzK2ZeRBC5ieGNg3aEymhFVerruRBFLjj%2FLy3UlvwN%2FhmumwzjYiZTVtalTOrsuMAAlQfYC0U0SYb%2FqceH9PNX05KpSF5gP1tkYucFawG9fUKxlpKws%2Bvo%2FvwnDxjyE5LC30N3u05%2B"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462bef788d814b4-HKG alt-svc: h3=":443"; ma=86400 [{"id":"eq-1","name":"Projector A","location":"Building 1"}, {"id":"eq-2","name":"Camera B","location":"Building 2"}, {"id":"eq-3","name":"Meeting Room C","location":"Building 3"}]</pre>

Test 2 — Create a booking (core)

Command	curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@ok.json"
Request body	File: ok.json
Expected	201 Created — Copy the returned id and save it into \$ID1
Screenshot	 <pre>PS D:\project\booking-api> curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@ok.json" HTTP/1.1 201 Created Date: Tue, 06 Oct 2026 07:05:23 GMT Content-Type: application/json Content-Length: 201 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=cPYsZDa4x4nk4%2BfTyoi4WAXhHyTYt77d86SfmTsh%2BwlzpbnoXcbI7fZQfE%2Fz8GoyGpMcmVC8DMtFSTJ90dPqkuv47ETSBkI1SdtxdwL6Lob8tI0CykrBjz0PDIPZhh8HPp7x7uoUkwTCwzfgd4AYC2cr%2BAUX%2BMIA6"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462d13b7ca8fd03-SIN alt-svc: h3=":443"; ma=86400 {"id":"eeee642b-2f43-4365-a073-f5fb96a1e618","equipmentId":"eq-1","borrowerName":"Somchai Jaidee","startAt":"2026-10-20T09:00:00.000Z","endAt":"2026-10-20T11:00:00.000Z","purpose":"Class presentation"} PS D:\project\booking-api> \$ID1 = "eeee642b-2f43-4365-a073-f5fb96a1e618"</pre>

Test 3 — List bookings

Command	curl.exe -i "\$BASE/bookings"
Expected	200 OK

Screenshot	<pre>PS D:\project\booking-api> curl.exe -i "\$BASE/bookings" HTTP/1.1 200 OK Date: Tue, 06 Oct 2026 07:07:49 GMT Content-Type: application/json Content-Length: 203 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=Uhc5MxISWPEEvegSKY0%2F2p%2FpPymnRlFYnmwoKQ2mAZtPD6lClDjj22fsN0hGwHENVpesA%2Bm14QlisMrKacYOYzGADne4cEJhxEj7XzsITmAz8C803kl9ugz1JJvzzuJdgAkW5hse6nRcdmAmgqLNE%2BprqRXhXi"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462d4d4482a49d1-SIN alt-svc: h3=":443"; ma=86400 [{"id":"eeea642b-2f43-4365-a073-f5fb96a1e618","equipmentId":"eq-1","borrowerName":"Somchai Jaidee","startAt":"2026-10-20T09:00:00Z","endAt":"2026-10-20T11:00:00.000Z","purpose":"Class presentation"}]</pre>
------------	---

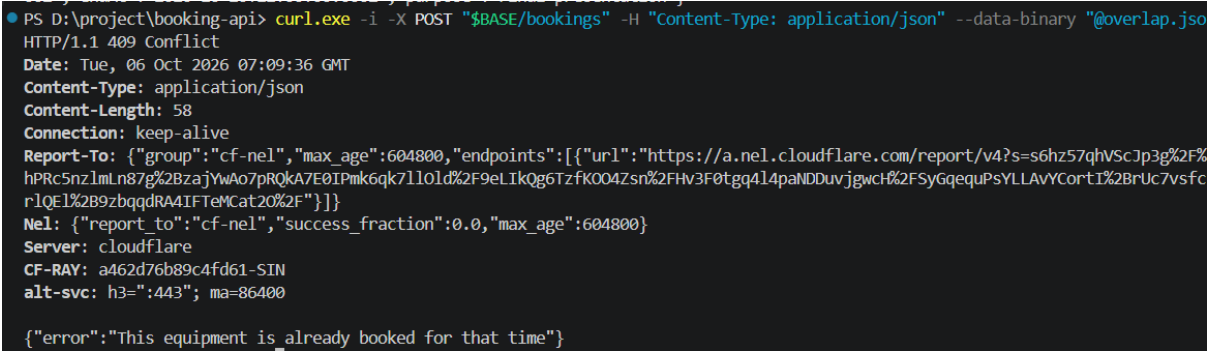
Test 4 — Get one booking (core)

Command	<code>curl.exe -i "\$BASE/bookings/\$ID1"</code>
Expected	200 OK(\$ID1 SAVED FROM TEST 2 IS REUSED)
Screenshot	<pre>PS D:\project\booking-api> curl.exe -i "\$BASE/bookings/\$ID1" HTTP/1.1 200 OK Date: Tue, 06 Oct 2026 07:08:15 GMT Content-Type: application/json Content-Length: 201 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=gFVys50mD%2FD5cAW5B8ja%2B3jwYn5EHLz8yXy%2BymLTolpwvkCO2G97cNibwT36YhcdQ3ExOf2L59P7Ddri%2BjZXOV%2FFWmqCLDIHLdTA%2B4w2aekF14VoPV8tvRqAeJaDR3eBMk8wsyHG14M9HjiUgMTsef%2FzPaZTKBmJ44H%2B"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462d576eadcf95-SIN alt-svc: h3=":443"; ma=86400 {"id":"eeea642b-2f43-4365-a073-f5fb96a1e618","equipmentId":"eq-1","borrowerName":"Somchai Jaidee","startAt":"2026-10-20T09:00:00.000Z","endAt":"2026-10-20T11:00:00.000Z","purpose":"Class presentation"}</pre>

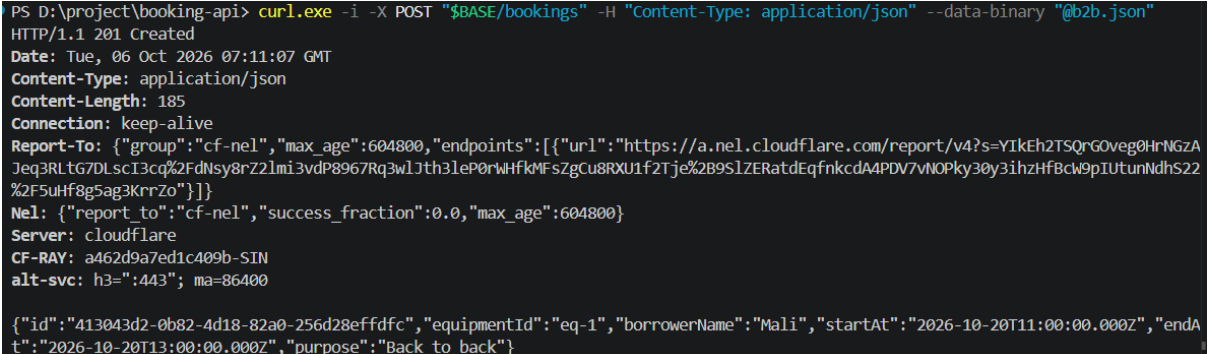
Test 5 — Update a booking (PATCH) (core)

Command	<code>curl.exe -i -X PATCH "\$BASE/bookings/\$ID1" -H "Content-Type: application/json" --data-binary "@patch.json"</code>
Request body	<i>File: patch.json</i>
Expected	200 OK — Response shows the changed purpose.
Screenshot	<pre>PS D:\project\booking-api> curl.exe -i -X PATCH "\$BASE/bookings/\$ID1" -H "Content-Type: application/json" --data-binary "@patch.json" HTTP/1.1 200 OK Date: Tue, 06 Oct 2026 07:09:05 GMT Content-Type: application/json Content-Length: 201 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=KNkgrEm7AAWkqapzOZXreE9TZBeTdzxChA0%2FvVfXmkv0IwILzWpX%2F6rVX%2BjIy0l126%2FBlrcfy2q%2BHB04wRyOwMuq6zOANCLJH5enZlw3n5ppummiHouynR%2BwevxEvFUqxS%2FpChXApeSYhtQDZ3BKmDJDthCCYSJ12I%2FP"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462d6a999b9d7ad-SIN alt-svc: h3=":443"; ma=86400 {"id":"eeea642b-2f43-4365-a073-f5fb96a1e618","equipmentId":"eq-1","borrowerName":"Somchai Jaidee","startAt":"2026-10-20T09:00:00.000Z","endAt":"2026-10-20T11:00:00.000Z","purpose":"Final presentation"}</pre>

Test 6 — Overlapping booking is rejected (core)

Command	<code>curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@overlap.json"</code>
Request body	<i>File: overlap.json</i>
Expected	409 Conflict
Screenshot	

Test 7 — Back-to-back booking is allowed

Command	<code>curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@b2b.json"</code>
Request body	<i>File: b2b.json</i> <code>{"equipmentId": "eq-1", "borrowerName": "Mali", "startAt": "2026-10-20T11:00:00.000Z", "endAt": "2026-10-20T13:00:00.000Z", "purpose": "Back to back"}</code>
Expected	201 Created — Copy the returned id into \$ID2.
Screenshot	

Test 8 — Update into a conflict

Command	<code>curl.exe -i -X PATCH "\$BASE/bookings/\$ID2" -H "Content-Type: application/json" --data-binary "@patchconflict.json"</code>
Request body	<i>File: patchconflict.json</i> <code>{"startAt": "2026-10-20T10:00:00.000Z"}</code>
Expected	409 Conflict(\$ID2 USED FROM TEST 7)

Screenshot	<pre> PS D:\project\booking-api> \$ID2 = "413043d2-0b82-4d18-82a0-256d28effdfc" PS D:\project\booking-api> curl.exe -i -X PATCH "\$BASE/bookings/\$ID2" -H "Content-Type: application/json" --data-binary "@patchconflict.json" HTTP/1.1 409 Conflict Date: Tue, 06 Oct 2026 07:13:12 GMT Content-Type: application/json Content-Length: 58 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=ST3hiXrbPJR0Xi5ELtrcDFqMeoDFCPxXdW0d9YBiczyVFdodKRUMWZeEfkyrGYtiogY7JanbIVj0EP1Qrcpr7bb28FIw%2FJUqlCHEswKpX9iS4YD60AfJc6rbXeN7ndXB5eFgmAmEg0D0GQQQcmI6ZShwk%2B2qj2SgfdAV"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462dcb57e699c59-SIN alt-svc: h3=":443"; ma=86400 {"error":"This equipment is already booked for that time"} PS D:\project\booking-api> </pre>
------------	---

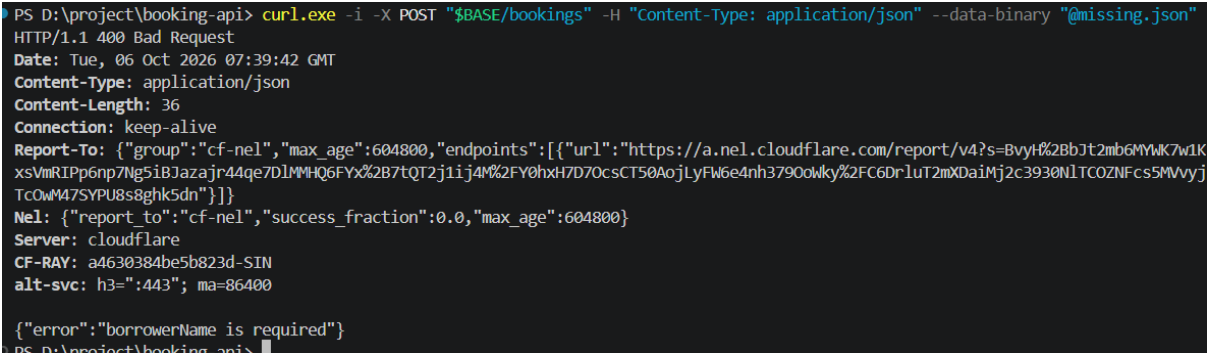
Test 9 — Start time after end time (core)

Command	curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@badtime.json"
Request body	File: badtime.json
Expected	400 Bad Request
Screenshot	<pre> PS D:\project\booking-api> curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@badtime.json" HTTP/1.1 400 Bad Request Date: Tue, 06 Oct 2026 07:14:01 GMT Content-Type: application/json Content-Length: 40 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=XSu%2FFil%2FEeyML%2FXbCsD6KF2iRgWdORnlKEezOne9cOn%2BC5rx%2FkwlVthSnSFst%2BiSOvL6YbnSwJ6VK8CYs59B6vDXknk%2BiW1yqYmJXnaUKIHz%2F77M%2Fci3e%2F19AQ1mw44FtbCN1kvgo%2FoPLw9p%2BMqR2D%2B5wrZup7fd"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462dde8eae6da84-HKG alt-svc: h3=":443"; ma=86400 {"error":"startAt must be before endAt"} </pre>

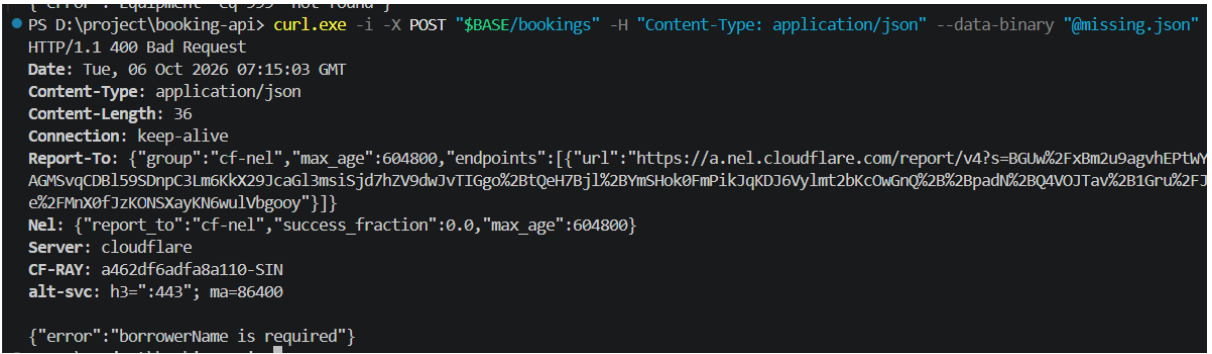
Test 10 — Unknown equipmentId (core)

Command	curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@noequip.json"
Request body	File: noequip.json
Expected	400 Bad Request
Screenshot	<pre> PS D:\project\booking-api> curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@noequip.json" HTTP/1.1 400 Bad Request Date: Tue, 06 Oct 2026 07:14:30 GMT Content-Type: application/json Content-Length: 40 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=Q1LePPoFQXRmk7o1DVjn6twvo%2FroRvsXw8AJJiw%2F52b6fETs6t7DYN0AgRMeinJFu7NuDD6gfJK2VtRIG3wGvWmrJsu0%2BP0AxlejrHpAgXs209ej1spGT3S9%2FB9dr%2BtjpcQe3g%2Fr0cB%2BguoFLLjESA5JUFRghK834Ms"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462de9d8b45f8ce-SIN alt-svc: h3=":443"; ma=86400 {"error":"Equipment 'eq-999' not found"} </pre>

Test 11 — Missing borrowerName

Command	<code>curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@missing.json"</code>
Request body	<i>File: missing.json</i> <code>{"equipmentId": "eq-1", "startAt": "2026-10-23T09:00:00.000Z", "endAt": "2026-10-23T10:00:00.000Z"}</code>
Expected	400 Bad Request
Screenshot	 <pre>PS D:\project\booking-api> curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@missing.json" HTTP/1.1 400 Bad Request Date: Tue, 06 Oct 2026 07:39:42 GMT Content-Type: application/json Content-Length: 36 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=BvyH%2BbJt2mb6MYWk7w1KxsVmRIPp6np7Ng5iBJazajr44qe7DlMMHQ6FYx%2B7tQT2j1ij4M%2FY0hxH7D70csCT50AojLyFw6e4nh3790owky%2FC6Dr1uT2mxDaiMj2c3930N1TC0ZNFcs5MvvyjTc0wM47SYPu8s8ghk5dn"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a4630384be5b823d-SIN alt-svc: h3=":443"; ma=86400 {"error":"borrowerName is required"} PS D:\project\booking-api></pre>

Test 12 — Invalid JSON

Command	<code>curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@badjson.json"</code>
Request body	<i>File: badjson.json</i> <code>{ this is not json</code>
Expected	400 Bad Request
Screenshot	 <pre>PS D:\project\booking-api> curl.exe -i -X POST "\$BASE/bookings" -H "Content-Type: application/json" --data-binary "@missing.json" HTTP/1.1 400 Bad Request Date: Tue, 06 Oct 2026 07:15:03 GMT Content-Type: application/json Content-Length: 36 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=BGUw%2FxBm2u9agvhEptWYAGMSvqCDBl59SDnpC3Lm6KkX29JcaG13msIsjd7hZV9dwJvTIGgo%2BtQeH7Bj1%2BYmShok0FmPikJqKDJ6Vylmt2bKcOwGnQ%2B%2BpadN%2BQ4VOJTav%2B1Gru%2FJe%2FmX0fJzKONSXayKN6wulVbgooy"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462df6adfa8a110-SIN alt-svc: h3=":443"; ma=86400 {"error":"borrowerName is required"} PS D:\project\booking-api></pre>

Test 13 — Booking not found

Command	<code>curl.exe -i "\$BASE/bookings/does-not-exist"</code>
Expected	404 Not Found

Screenshot	<pre>PS D:\project\booking-api> curl.exe -i "\$BASE/bookings/does-not-exist" HTTP/1.1 404 Not Found Date: Tue, 06 Oct 2026 07:15:35 GMT Content-Type: application/json Content-Length: 29 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=zjx95cx6uqchj0yt3YBSSzDfFPGfVvBws80WJRY03EpA%2FO20S6mx544uhYoPZcRcA9w1MvKY8wGxhVNZ4sE%2B3UkYVQ86Vs9GoPL1Dx%2BZeGoOq62mNG0j2MQeG22bJ5%2B%2BqTUa0xiMce%2B4CB3cNaOrZL9G6y1LEwG9p1v"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462e02f5ccef4fb-HKG alt-svc: h3=":443"; ma=86400 {"error":"Booking not found"}</pre>
------------	---

Test 14 — Delete a booking (core)

Command	<code>curl.exe -i -X DELETE "\$BASE/bookings/\$ID1"</code>
Expected	204 No Content — No response body is correct for 204.
Screenshot	<pre>PS D:\project\booking-api> curl.exe -i -X DELETE "\$BASE/bookings/\$ID1" HTTP/1.1 204 No Content Date: Tue, 06 Oct 2026 07:15:54 GMT Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=%2BpfzhKLqeG1lbJ%2B4ipTxx1AaIrH7yEEOGJTBT7NeQRorIX%2FfZa6e1EBQXDhq6xqVat%2F4si%2Bml%2BE%2FmQEEd1BuwSgjqxtrLFrk%2BvPoyeZ%2BpSjhn1pK6Tz75yWgpPvjs2jj8RuTzPglmrUzcJLKZzkXm2MfDqSqfINzM9F"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462e0ab88d38965-SIN alt-svc: h3=":443"; ma=86400</pre>

Test 15 — Get the deleted booking from Test 14 (core)

Command	<code>curl.exe -i "\$BASE/bookings/\$ID1"</code>
Expected	404 Not Found
Screenshot	<pre>PS D:\project\booking-api> curl.exe -i "\$BASE/bookings/\$ID1" HTTP/1.1 404 Not Found Date: Tue, 06 Oct 2026 07:16:17 GMT Content-Type: application/json Content-Length: 29 Connection: keep-alive Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=zVD%2BUYkhd8dJghXSRFONMpaALueChrTzGTUN6cFe5cfr12Y%2F8EoVKkhPCnUsoBWTuVzQC05HJIPYmJbJ7AfG6X97f2jggKNoC7sv1NtCZr1aPCAsp3knZc7m01bH7IuFowcT5MNDxp83tlc%2ce5krKvlieRYzPolPz"}]} Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800} Server: cloudflare CF-RAY: a462e13b1875fe86-SIN alt-svc: h3=":443"; ma=86400 {"error":"Booking not found"}</pre>

Cleanup

`curl.exe -i -X DELETE "$BASE/bookings/$ID2"`

```
PS D:\project\booking-api> curl.exe -i -X DELETE "$BASE/bookings/$ID2"
HTTP/1.1 204 No Content
Date: Tue, 06 Oct 2026 07:16:47 GMT
Connection: keep-alive
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=wnB57Gtbkiyeidg9X25c3kFaTStNb%2B%2FbvaZc9M6p4Cz9zJMRVU08Ap2ueuED9Po5qbqMNg0yctMw%2FxmBjku4UjyVLIithvkPWFVyx%2BdEtqlmsogV6thjuXI6YNEpe6Y3zzw7gb5CctFeu2wd6mq3X890%2FA6ocLbYw"}]}
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
Server: cloudflare
CF-RAY: a462e1f54f7040bc-SIN
alt-svc: h3=":443"; ma=86400
```

3. Code Evidence

3.1 Schema (schema.sql)

```
DROP TABLE IF EXISTS bookings;
DROP TABLE IF EXISTS equipment;

CREATE TABLE equipment (
  id    TEXT PRIMARY KEY,
  name  TEXT NOT NULL,
  location TEXT NOT NULL
);

CREATE TABLE bookings (
  id          TEXT PRIMARY KEY,
  equipmentId TEXT NOT NULL REFERENCES equipment(id),
  borrowerName TEXT NOT NULL,
  startAt    TEXT NOT NULL, -- ISO 8601 UTC
  endAt      TEXT NOT NULL, -- ISO 8601 UTC
  purpose    TEXT NOT NULL DEFAULT ""
);

CREATE INDEX idx_bookings_equipment_time ON bookings (equipmentId, startAt, endAt);

INSERT INTO equipment (id, name, location) VALUES
('eq-1', 'Projector A', 'Building 1'),
('eq-2', 'Camera B', 'Building 2'),
('eq-3', 'Meeting Room C', 'Building 3');
```

3.2 Parameter binding (src/index.ts)

All SQL that uses request data uses ? placeholders with .bind(). Request data is never added into SQL strings.

```

import { Hono } from "hono";

type Env = { Bindings: { DB: D1Database } };

type Booking = {
  id: string;
  equipmentId: string;
  borrowerName: string;
  startAt: string;
  endAt: string;
  purpose: string;
};

const app = new Hono<Env>().basePath("/api");

// ----- helpers -----
const err = (message: string) => ({ error: message });

// Turn any valid date string into a standard ISO string (or null if invalid)
function toIso(value: unknown): string | null {
  if (typeof value !== "string") return null;
  const d = new Date(value);
  return isNaN(d.getTime()) ? null : d.toISOString();
}

// Validate a full booking object. Returns an error message or the clean booking.
async function validate(
  db: D1Database,
  data: Record<string, unknown>,
  ignoreId?: string
): Promise<{ error: string; status: 400 | 404 | 409 } | { booking: Omit<Booking, "id"> }> {
  const { equipmentId, borrowerName, startAt, endAt, purpose } = data;

  if (typeof equipmentId !== "string" || !equipmentId.trim())
    return { error: "equipmentId is required", status: 400 };
  if (typeof borrowerName !== "string" || !borrowerName.trim())
    return { error: "borrowerName is required", status: 400 };
  if (purpose !== undefined && typeof purpose !== "string")
    return { error: "purpose must be a string", status: 400 };

  const start = toIso(startAt);
  const end = toIso(endAt);
  if (!start || !end)
    return { error: "startAt and endAt must be valid ISO date-times", status: 400 };
  if (start >= end) return { error: "startAt must be before endAt", status: 400 };

  // equipmentId must exist
  const eq = await db.prepare("SELECT id FROM equipment WHERE id = ?").bind(equipmentId).first();

```

```

if (!eq) return { error: `Equipment '${equipmentId}' not found`, status: 400 };

// overlap check: existing.start < new.end AND existing.end > new.start
const clash = await db
  .prepare(
    `SELECT id FROM bookings
     WHERE equipmentId = ? AND startAt < ? AND endAt > ? AND id != ?
     LIMIT 1`
  )
  .bind(equipmentId, end, start, ignoreId ?? "")
  .first();
if (clash)
  return { error: "This equipment is already booked for that time", status: 409 };

return {
  booking: {
    equipmentId,
    borrowerName: borrowerName.trim(),
    startAt: start,
    endAt: end,
    purpose: (purpose as string | undefined) ?? "",
  },
};
}

async function readJson(c: any): Promise<Record<string, unknown> | null> {
  try {
    const body = await c.req.json();
    return body && typeof body === "object" && !Array.isArray(body) ? body : null;
  } catch {
    return null;
  }
}

// ----- equipment -----
app.get("/equipment", async (c) => {
  const { results } = await c.env.DB.prepare("SELECT id, name, location FROM equipment").all();
  return c.json(results, 200);
});

// ----- bookings -----
app.get("/bookings", async (c) => {
  const equipmentId = c.req.query("equipmentId");
  const stmt = equipmentId
    ? c.env.DB.prepare("SELECT * FROM bookings WHERE equipmentId = ? ORDER BY startAt")
    : c.env.DB.prepare("SELECT * FROM bookings ORDER BY startAt");
  const { results } = await stmt.all();

```

```

    return c.json(results, 200);
  });

app.get("/bookings/:id", async (c) => {
  const row = await c.env.DB.prepare("SELECT * FROM bookings WHERE id = ?")
    .bind(c.req.param("id"))
    .first();
  if (!row) return c.json(err("Booking not found"), 404);
  return c.json(row, 200);
});

app.post("/bookings", async (c) => {
  const body = await readJson(c);
  if (!body) return c.json(err("Request body must be a valid JSON object"), 400);

  const result = await validate(c.env.DB, body);
  if ("error" in result) return c.json(err(result.error), result.status);

  const id = crypto.randomUUID();
  const b = result.booking;
  await c.env.DB.prepare(
    "INSERT INTO bookings (id, equipmentId, borrowerName, startAt, endAt, purpose) VALUES (?, ?, ?, ?, ?, ?)"
  )
    .bind(id, b.equipmentId, b.borrowerName, b.startAt, b.endAt, b.purpose)
    .run();

  return c.json({ id, ...b }, 201);
});

app.patch("/bookings/:id", async (c) => {
  const id = c.req.param("id");
  const existing = await c.env.DB.prepare("SELECT * FROM bookings WHERE id = ?").bind(id).first<Booking>();
  if (!existing) return c.json(err("Booking not found"), 404);

  const body = await readJson(c);
  if (!body) return c.json(err("Request body must be a valid JSON object"), 400);

  // merge changes into the existing booking, then validate the whole thing
  const merged = { ...existing, ...body };
  const result = await validate(c.env.DB, merged, id);
  if ("error" in result) return c.json(err(result.error), result.status);

  const b = result.booking;
  await c.env.DB.prepare(
    "UPDATE bookings SET equipmentId = ?, borrowerName = ?, startAt = ?, endAt = ?, purpose = ? WHERE id = ?"
  )
    .bind(b.equipmentId, b.borrowerName, b.startAt, b.endAt, b.purpose, id)

```

```

.run();

return c.json({ id, ...b }, 200);
});

app.delete("/bookings/:id", async (c) => {
  const id = c.req.param("id");
  const existing = await c.env.DB.prepare("SELECT id FROM bookings WHERE id = ?").bind(id).first();
  if (!existing) return c.json(err("Booking not found"), 404);
  await c.env.DB.prepare("DELETE FROM bookings WHERE id = ?").bind(id).run();
  return c.body(null, 204);
});

// ----- errors -----
app.notFound((c) => c.json(err("Route not found"), 404));
app.onError((e, c) => {
  console.error(e);
  return c.json(err("Internal server error"), 500);
});

export default app;

```

3.3 Overlap rule

A new booking conflicts with an existing one for the same equipment when `newStart < existingEnd` AND `newEnd > existingStart`. Because the comparison is strict, 09:00–11:00 followed by 11:00–13:00 is allowed (Test 7).